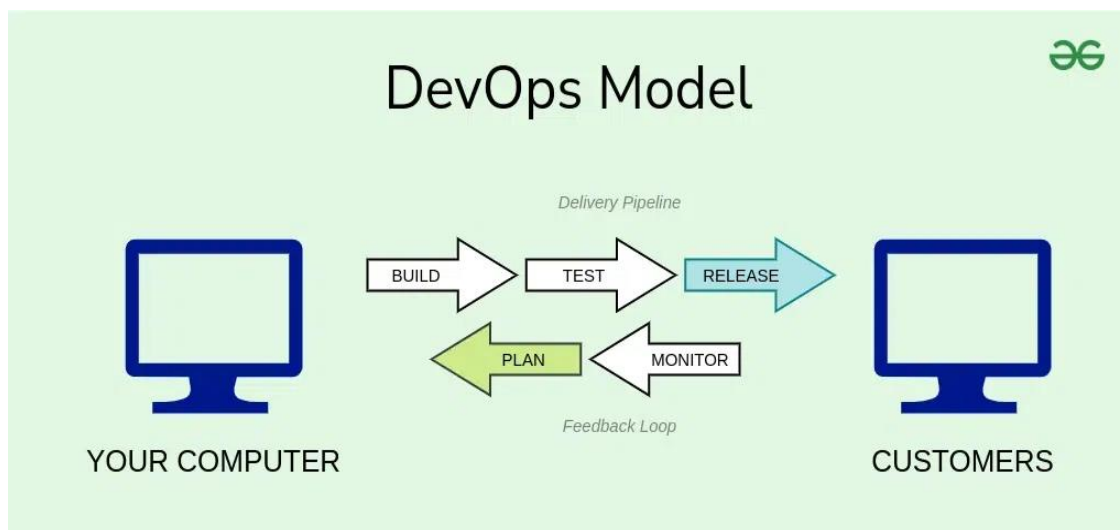


Unit-1

Introduction to DevOps: Introduction, Agile development model, DevOps and ITIL. DevOps process and Continuous Delivery, Release management, Scrum, Kanban, delivery pipeline, identifying bottlenecks

DevOps is a software development and IT methodology that combines people, processes, and technology to improve and speed up the software development life cycle. It's a combination of the words "development" (Dev) and "operations" (Ops).

DevOps is a philosophy that promotes better communication and collaboration between these teams and others in an organization. In its most narrow interpretation, DevOps describes the adoption of iterative software development, automation and programmable infrastructure deployment and maintenance. The term also covers culture changes, such as building trust and cohesion between developers and systems administrators and aligning technological projects to business requirements. DevOps can change the software delivery chain, services, job roles, IT tools and best practices.



Delivery Pipeline: The pipeline represents the different stages that software goes through before it is released to production. These stages might typically include:

- **Build:** The stage where the software code is compiled and packaged into a deployable unit.

- **Test:** The stage where the software is rigorously tested to ensure it functions as expected and identifies any bugs.
- **Release:** The stage where the software is deployed to production for end users.

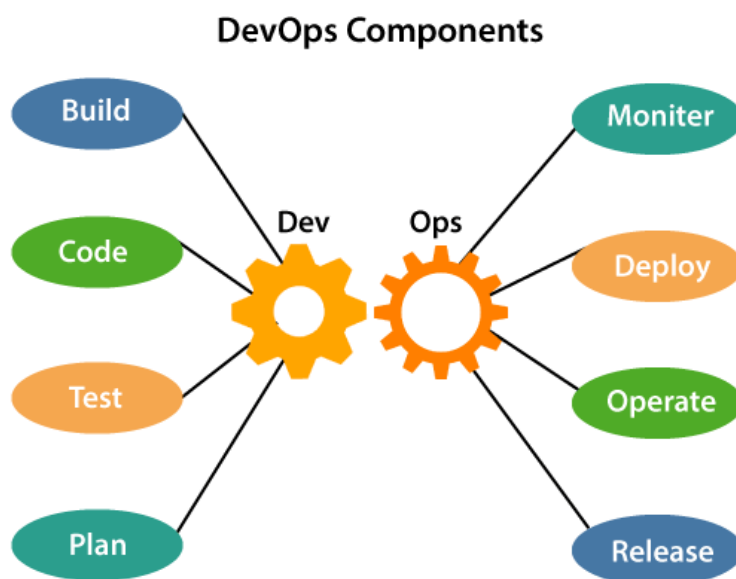
How does DevOps work?

A DevOps team includes developers and IT operations working collaboratively throughout the product lifecycle, in order to increase the speed and quality of software deployment. It's a new way of working, a cultural shift, that has significant implications for teams and the organizations they work for.

Under a DevOps model, development and operations teams are no longer “siloesd.” Sometimes, these two teams merge into a single team where the engineers work across the entire application lifecycle — from development and test to deployment and operations — and have a range of multidisciplinary skills.

DevOps teams use tools to automate and accelerate processes, which helps to increase reliability. A DevOps toolchain helps teams tackle important DevOps fundamentals including continuous integration, continuous delivery, automation, and collaboration.

DevOps values are sometimes applied to teams other than development. When security teams adopt a DevOps approach, security is an active and integrated part of the development process.



1) Build

Without DevOps, the cost of the consumption of the resources was evaluated based on the pre-defined individual usage with fixed hardware allocation. And with DevOps, the usage of cloud, sharing of resources comes into the picture, and the build is dependent upon the user's need, which is a mechanism to control the usage of resources or capacity.

2) Code

Many good practices such as Git enables the code to be used, which ensures writing the code for business, helps to track changes, getting notified about the reason behind the difference in the actual and the expected output, and if necessary reverting to the original code developed. The code can be appropriately arranged in **files**, **folders**, etc. And they can be reused.

3) Test

The application will be ready for production after testing. In the case of manual testing, it consumes more time in testing and moving the code to the output. The testing can be automated, which decreases the time for testing so that the time to deploy the code to production can be reduced as automating the running of the scripts will remove many manual steps.

4) Plan

DevOps use Agile methodology to plan the development. With the operations and development team in sync, it helps in organizing the work to plan accordingly to increase productivity.

5) Monitor

Continuous monitoring is used to identify any risk of failure. Also, it helps in tracking the system accurately so that the health of the application can be checked. The monitoring becomes more comfortable with services where the log data may get monitored through many third-party tools such as **Splunk**.

6) Deploy

Many systems can support the scheduler for automated deployment. The cloud management platform enables users to capture accurate insights and view the optimization scenario, analytics on trends by the deployment of dashboards.

7) Operate

DevOps changes the way traditional approach of developing and testing separately. The teams operate in a collaborative way where both the teams actively participate throughout the service lifecycle. The operation team interacts with developers, and they come up with a monitoring plan which serves the IT and business requirements.

8) Release

Deployment to an environment can be done by automation. But when the deployment is made to the production environment, it is done by manual triggering. Many processes involved in release management commonly used to do the deployment in the production environment manually to lessen the impact on the customers.

DevOps Lifecycle

DevOps Lifecycle is the set of phases that includes [DevOps](#) for taking part in [Development](#) and Operation group duties for quicker software program delivery. DevOps follows positive techniques that consist of **code, building, testing, releasing, deploying, operating, displaying, and planning**. **DevOps lifecycle** follows a range of phases such as non-stop development, non-stop integration, non-stop testing, non-stop monitoring, and non-stop feedback. Each segment of the DevOps lifecycle is related to some equipment and applied sciences to obtain the process. Some of the frequently used tools are open source and are carried out primarily based on commercial enterprise requirements. DevOps lifecycle is effortless to manipulate and it helps satisfactory delivery.

7 Cs of DevOps

1. Continuous Development
2. Continuous Integration
3. Continuous Testing
4. Continuous Deployment/Continuous Delivery
5. Continuous Monitoring
6. Continuous Feedback
7. Continuous Operations



The DevOps lifecycle includes seven phases as given below:

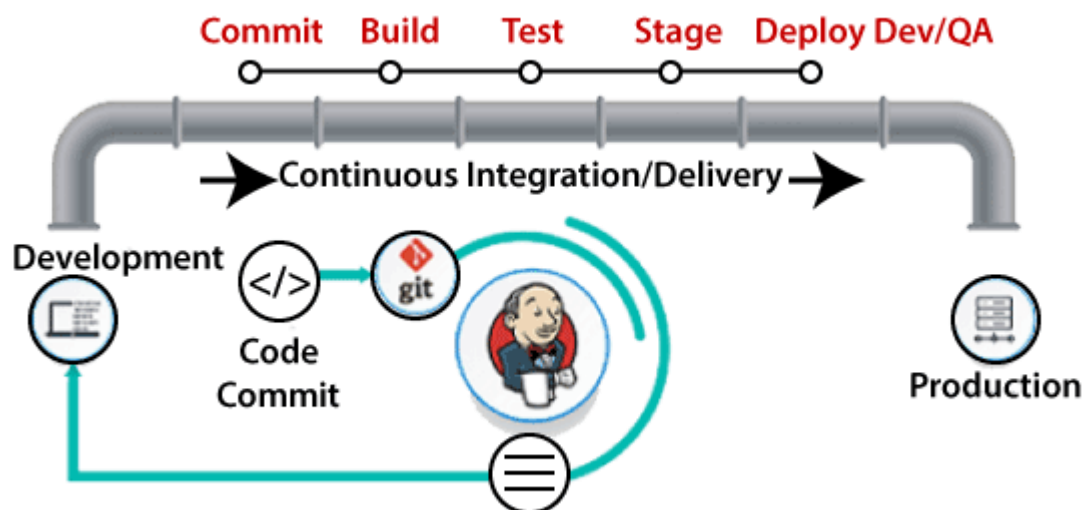
1) Continuous Development

This phase involves the planning and coding of the software. The vision of the project is decided during the planning phase. And the developers begin developing the code for the application. There are no DevOps tools that are required for planning, but there are several tools for maintaining the code.

2) Continuous Integration

This stage is the heart of the entire DevOps lifecycle. It is a software development practice in which the developers require to commit changes to the source code more frequently. This may be on a daily or weekly basis. Then every commit is built, and this allows early detection of problems if they are present. Building code is not only involved compilation, but it also includes **unit testing, integration testing, code review, and packaging.**

The code supporting new functionality is continuously integrated with the existing code. Therefore, there is continuous development of software. The updated code needs to be integrated continuously and smoothly with the systems to reflect changes to the end-users.



Jenkins is a popular tool used in this phase. Whenever there is a change in the Git repository, then Jenkins fetches the updated code and prepares a build of that code, which is an executable file in the form of war or jar. Then this build is forwarded to the test server or the production server.

3) Continuous Testing

This phase, where the developed software is continuously testing for bugs. For constant testing, automation testing tools such as **TestNG**, **JUnit**, **Selenium**, etc are used. These tools allow QAs to test multiple code-bases thoroughly in parallel to ensure that there is no flaw in the functionality. In this phase, **Docker** Containers can be used for simulating the test environment.



Selenium does the automation testing, and TestNG generates the reports. This entire testing phase can automate with the help of a Continuous Integration tool called **Jenkins**.

Automation testing saves a lot of time and effort for executing the tests instead of doing this manually. Apart from that, report generation is a big plus. The task of evaluating the test cases that failed in a test suite gets simpler. Also, we can

schedule the execution of the test cases at predefined times. After testing, the code is continuously integrated with the existing code.

4) Continuous Monitoring

Monitoring is a phase that involves all the operational factors of the entire DevOps process, where important information about the use of the software is recorded and carefully processed to find out trends and identify problem areas. Usually, the monitoring is integrated within the operational capabilities of the software application.

It may occur in the form of documentation files or maybe produce large-scale data about the application parameters when it is in a continuous use position. The system errors such as server not reachable, low memory, etc are resolved in this phase. It maintains the security and availability of the service.

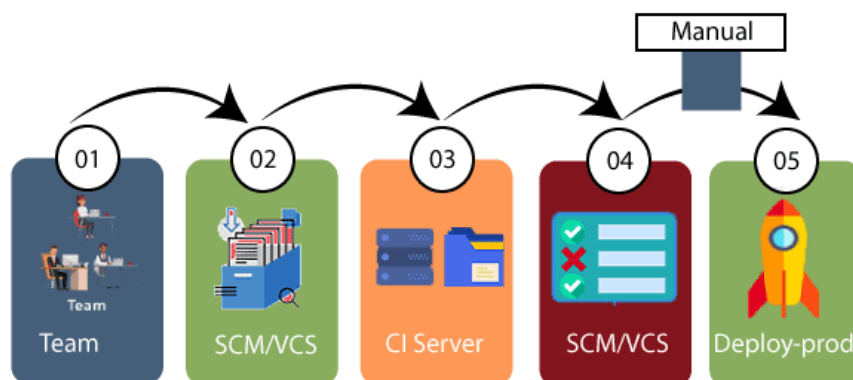
5) Continuous Feedback

The application development is consistently improved by analyzing the results from the operations of the software. This is carried out by placing the critical phase of constant feedback between the operations and the development of the next version of the current software application.

The continuity is the essential factor in the DevOps as it removes the unnecessary steps which are required to take a software application from development, using it to find out its issues and then producing a better version. It kills the efficiency that may be possible with the app and reduce the number of interested customers.

6) Continuous Deployment

In this phase, the code is deployed to the production servers. Also, it is essential to ensure that the code is correctly used on all the servers.



The new code is deployed continuously, and configuration management tools play an essential role in executing tasks frequently and quickly. Here are some popular tools which are used in this phase, such as **Chef**, **Puppet**, **Ansible**, and **SaltStack**.

Containerization tools are also playing an essential role in the deployment phase. **Vagrant** and **Docker** are popular tools that are used for this purpose. These tools help to produce consistency across development, staging, testing, and production environment. They also help in scaling up and scaling down instances softly.

Containerization tools help to maintain consistency across the environments where the application is tested, developed, and deployed. There is no chance of errors or failure in the production environment as they package and replicate the same dependencies and packages used in the testing, development, and staging environment. It makes the application easy to run on different computers.

7) Continuous Operations

All DevOps operations are based on the continuity with complete automation of the release process and allow the organization to accelerate the overall time to market continuingly.

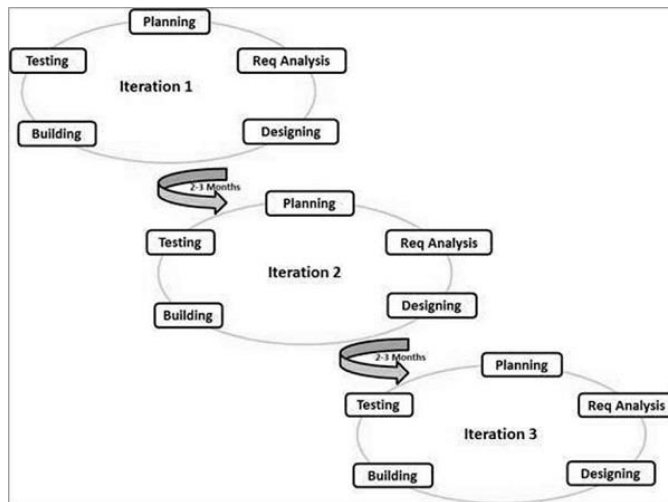
It is clear from the discussion that continuity is the critical factor in the DevOps in removing steps that often distract the development, take it longer to detect issues and produce a better version of the product after several months. With DevOps, we can make any software product more efficient and increase the overall count of interested customers in your product.

Agile Development Model

Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In Agile, the tasks are divided to time boxes (small time frames) to deliver specific features for a release.

Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the final build holds all the features required by the customer.

Here is a graphical illustration of the Agile Model-



The Agile thought process had started early in the software development and started becoming popular with time due to its flexibility and adaptability.

The most popular Agile methods include Rational Unified Process (1994), Scrum (1995), Crystal Clear, Extreme Programming (1996), Adaptive Software Development, Feature Driven Development, and Dynamic Systems Development Method (DSDM) (1995). These are now collectively referred to as **Agile Methodologies**, after the Agile Manifesto was published in 2001.

Following are the Agile Manifesto principles –

- **Individuals and interactions** – In Agile development, self-organization and motivation are important, as are interactions like co-location and pair programming.
- **Working software** – Demo working software is considered the best means of communication with the customers to understand their requirements, instead of just depending on documentation.
- **Customer collaboration** – As the requirements cannot be gathered completely in the beginning of the project due to various factors, continuous customer interaction is very important to get proper product requirements.
- **Responding to change** – Agile Development is focused on quick responses to change and continuous development.

The advantages of the Agile Model are as follows –

- Is a very realistic approach to software development.
- Promotes teamwork and cross training.
- Functionality can be developed rapidly and demonstrated.

- Resource requirements are minimum.
- Suitable for fixed or changing requirements
- Delivers early partial working solutions.
- Good model for environments that change steadily.

The disadvantages of the Agile Model are as follows –

- Not suitable for handling complex dependencies.
- More risk of sustainability, maintainability and extensibility.
- An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- There is a very high individual dependency, since there is minimum documentation generated.

What is ITIL?

ITIL involves using a set of best practices, which are designed to elevate your ITSM, improve customer relations, and establish a reliable environment for optimal delivery of IT services.

Pros

- Streamlines IT service delivery
- Increases customer satisfaction
- Allows for better ITSM integration
- Helps align IT goals with company objectives
- Optimizes IT usage

Cons

- Less flexibility
- Can be difficult to learn

- Costly to implement
- Can be challenging to integrate if you're unfamiliar with ITIL

Difference between DevOps and ITIL :

S.NO.	DevOps	ITIL
01.	DevOps refers to effective collaboration between the development team and operations team.	ITIL refers to a set of detailed guidelines for effective and efficient management of an organization's IT services.
02.	Use a methodical approach to minimize the friction between two teams.	It uses a systematic approach to manage the IT service to ensure growth.
03.	CI and CD are the backbone of modern DevOps philosophy.	ITIL aims to increase the delivery process.
04.	Continuous integration and continuous delivery are critical to increasing.	Services are built, discuss, tested, and implemented.
05.	DevOps focus on the concept. It has a dynamic body of knowledge.	DevOps focus on the concept. It has a dynamic body of knowledge. ITIL focus on development. It has a static body of knowledge.

DevOps Process

DevOps Process

DevOps is a set of practices that enhances collaboration between development (Dev) and operations (Ops) teams to improve software delivery and performance.

DevOps Process involves :-

Collaboration: Encourages communication and cooperation between Dev and Ops teams.

Automation: Streamlines tasks like testing, deployment, and monitoring.

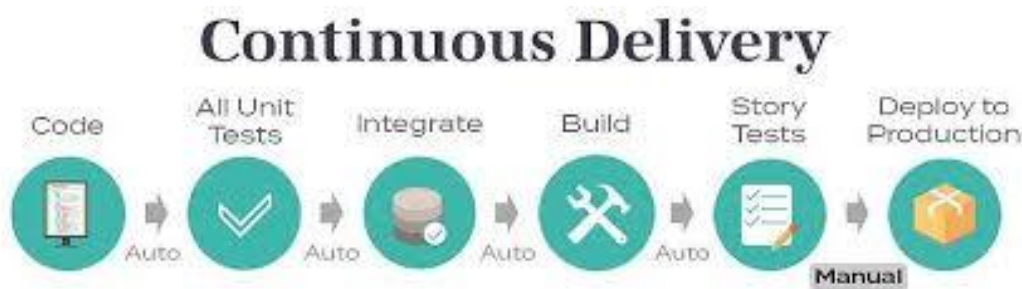
Continuous Integration (CI): Frequently integrates code changes and runs automated tests.

Continuous Delivery (CD): Automates the release process to ensure code can be deployed to production at any time.

Monitoring and Feedback: Uses monitoring and logging to gather feedback and improve systems.

Continuous Delivery(CD)

Continuous delivery (CD) is a DevOps practice that automates the process of building, testing, configuring, and deploying software to production. The goal of CD is to ensure that a codebase is always ready to be deployed, and that it's easy to deploy new code.



Here are some key aspects of continuous delivery:

- **Automated testing:** CD automates testing beyond unit tests, including UI testing, load testing, integration testing, and API reliability testing. This helps developers find issues before deploying updates.
- **Release pipeline:** A release pipeline can create multiple testing or staging environments to automate infrastructure creation and deploy new builds.
- **Blue/green deployment:** A common method for CD, this technique involves configuring two identical environments. One environment serves end users, while the other is ready for new code to be deployed and tested.
- **Deployment frequency:** In theory, CD allows you to deploy as often as you like, but it's recommended to deploy to production early and release small batches. This makes it easier to troubleshoot problems.

Release Management

What is Release Management?

Release management is the process of overseeing the planning, scheduling, and controlling of software builds throughout each stage of development and across various environments. Release management typically included the testing and deployment of software releases as well.

Release management has had an important role in the software development lifecycle since before it was known as release management. Deciding when and how to release updates was its own unique problem even when software saw physical disc releases with updates occurring as seldom as every few years.

Now that most software has moved from hard and fast release dates to the software as a service (SaaS) business model, release management has become a constant process that works alongside development. This is especially true for businesses that have converted to utilizing continuous delivery pipelines that see new releases occurring at blistering rates. DevOps now plays a large role in many of the duties that were originally considered to be under the purview of release management roles; however, DevOps has not resulted in the obsolescence of release management.

Advantages of Release Management for DevOps

1. Streamlined Deployment.
2. Reduced Risk.
3. Faster Time-to-Market
4. Improved Quality
5. Enhanced Collaboration

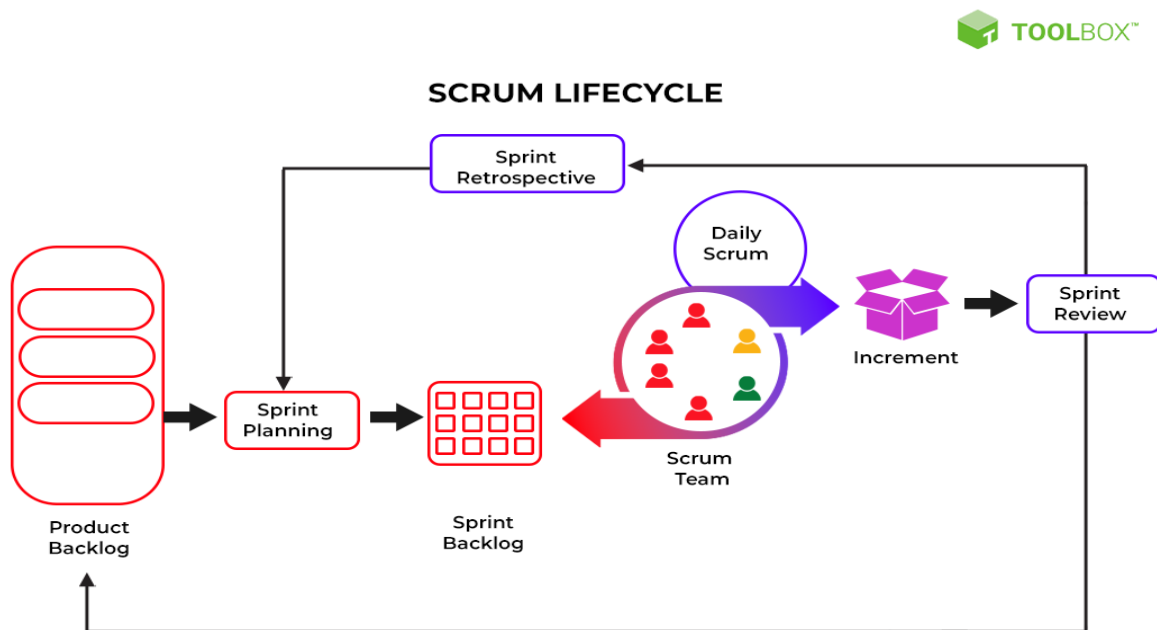
Scrum

What is Scrum?

Scrum is a variation of Agile Methodology implementation that allows products to be built in a highly-volatile environment with development requests changing frequently. This process leverages short development cycles called sprints which last from two weeks to a month at max.

These teams are small and have between 3 to 9 members, including a scrum master and a product owner.

The process encourages constant communication between its members and seeks to maintaining the flow of development using ceremonies like the Daily Scrum (which is like a daily standup to discuss blockers, etc); sprint reviews (to demonstrate the work done to stakeholders) and sprint retrospectives (to understand what worked well and what parts to correct).



Benefits of Scrum

The many benefits of Scrum include:

1. Helping teams achieve project deliverables quickly and efficiently while working in a volatile and agile environment
2. Helps to divide large projects into easily manageable sprints.
3. The scrum ceremonies provide transparency for all stakeholders involved, and help to highlight any blockers
4. Short sprints are more manageable and can be customised to react to change in requirements.

Limitations of Scrum

No methodology, however efficient, can provide a silver bullet to the organisation's problems. Some limitations of Scrum being:

1. The propensity for scope creep, due to the lack of a definite end-date
2. Challenges in adopting scrum for large teams as it becomes difficult to have many people on the same page
3. There are risks when team members leave in between a Scrum process as it negatively impacts the entire cycle.
4. Product quality often takes a hit as the focus on smaller sprints is on code delivery and there may not be enough time or resources to effectively maintain a robust QA process.

Kanban

Kanban methodology is an agile method that aims at continuous improvement, flexibility in [task management](#), and enhanced workflow. With this illustrative approach, the progress of the whole project can be easily understood in a glance.

Kanban was used in manufacturing settings to control inventory throughout the supply chain, using a practice called just-in-time (JIT) manufacturing. In [project management](#), the Kanban methodology adapts the same concept by ensuring that the amount of required work is the same as the work capabilities of the team.

How does Kanban work?

Kanban method revolves around the [kanban board](#). It is a tool that visualizes the entire project to track the flow of their project. Through this graphical approach of Kanban boards, a new member or an external entity can understand what's happening right now, tasks completed and future tasks.

Kanban board indicates:-

- the current tasks that are being performed
- the tasks to do in the future
- the tasks that are completed



Principles of Kanban

Kanban is based on four key principles which are mentioned below:

1. **Start with the existing process:** It is a change management method that starts with the existing process. Changes are done in the system in incremental and evolutionary ways. Unlike [Scrum](#), there's no specific process or roles defined in Kanban.
2. **Agree to continue evolutionary and incremental changes:** After starting with the existing process, the team must agree on continuous, incremental, and evolutionary changes. The changes should be small and incremental. Rapid and substantial changes may be effective but they will be subjected to larger resistance as well by the Team.
3. **Admire current roles, processes, responsibilities & titles:** Though Kanban suggests continuous incremental changes in the process, it respects current roles, responsibilities, and job titles. This helps the team to gain confidence as they get started with Kanban.
4. **Leadership at all levels:** Kanban does not expect leadership from a specific set, rather the actions of leadership at all levels in the organization, are very much encouraged.

Kanban Practices

The following are the six core Kanban practices:

1. **Limit WIP:** Limiting Work-In-Process (WIP) implies that a pull system is executed on either parts or the whole workflow. It (PULL system) will act as one of the key stimuli for incremental, continuous, and evolutionary changes to the system. Limit WIP assigns explicit limits to the number of items that may be in progress at each workflow state.
2. **Visualize:** Visualizing the workflow and making it visible is important so as to know how work proceeds. Without understanding the flow of work, incorporating the right changes is difficult. Usually, a card wall with

columns and cards is used to visualize the flow of work. Different states or steps within the workflow are represented by the columns on the card wall.

3. **Manage flow:** Flow of work through every state within the workflow should be observed, measured, and informed. By managing the flow vigorously, the incremental, continuous, and evolutionary modifications to the system can be assessed to have negative or positive effects on the system.
4. **Improve Collaboratively, Evolve Experimentally:** Kanban encourages small incremental, continuous, and evolutionary changes. Whenever teams have a common understanding of concepts about work, process, workflow, and risk, they are more likely to be able to form a shared understanding of a problem and suggest enhancement actions that could achieve a consensus.
5. **Implement Feedback Loops:** Early feedback from clients and the pull system are important in Kanban. If we get feedback from different stakeholders and processes, it will help to eliminate risk and optimize the delivery process.
6. **Make Policies Explicit:** Until the mechanism of a process is not made clear, it is difficult to hold a debate and discuss ways to improve it. Without a clear understanding of how work is truly done and how things actually work, any conversation of complications tends to be anecdotal, emotional, and subjective. With a clear understanding, it is possible to hold a more rational, empirical, objective discussion of issues. It is more likely to facilitate consensus around improvement suggestions.

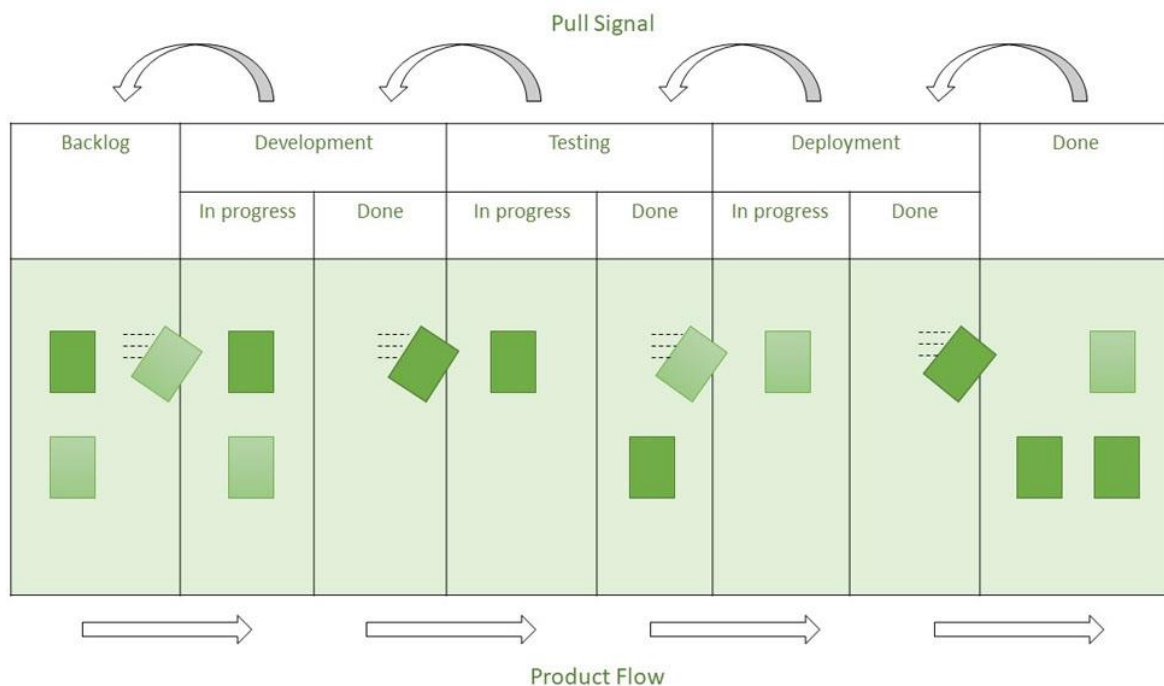
Kanban Workflow – How does Kanban Pull System Work?

Step 1. Visualize your Workflow

- Identify work stages and the work items.
- Work items: The max effort of 2 days to keep it short and moving.
 - Write work items on cards and stick them in columns, under corresponding work stages, based on system workflow – From left to right.

- Workflow can be depicted physically or with a tool like Jira.
- Sample Work stages: Design, Development, Test, Production, Deployment, Done.
 - Each Stage can be split into- Ready and In Progress.
 - Initially, these two sub-stages may not be present.
 - Later they are introduced to check the waiting time in each stage.

Step 2. Establish a Pull System



- Rather than pushing tasks into the process, teams pull work only when there is a need for it and they have the capacity to meet it. 'In progress (IP)' and 'Done' columns appear in every process state.
- For example, when a development task is finished, it is moved to the 'Development (Done)' column.
- Queue statuses are what 'Done' columns are referred to be. No one works on tasks in the queue states; hence these are passive.
- In our example, 'Development (Done)', is the queue from which testing teams pull tasks ready for testing once they have the capacity to handle new work.

- Teams can avoid multitasking by pulling work and focusing on the most important tasks.
- This results in higher throughput and shorter cycle times, which implies happy customers and higher profits.

Step 3. Limiting WIP

Create a Pull system upstream by setting WIP limits, and also ensure that the team “Stops beginning and Starts Finishing.” After finishing a task in the current stage, team members can pull a task from the previous stage, thereby freeing up capacity in the preceding stage. This keeps going till the input of the bugs/ tickets.

- WIP limits could be set based on historical data and capacity planning.
- In case many tickets get piled up in one stage other team members help their team members so that tickets movement is smooth, thereby increasing collaboration.
- WIP limits defined could be observed for 3 to 4 weeks and updated based on the team’s experience.

The sample Kanban board with WIP limits introduced is given below:

Workstage:	Backlog	Acknowledged	Development		Testing		Deployment	Done
WIP Limit			5		3		2	
Status	-	-	In progress	Ready	In progress	Ready	-	-
Job Cards								

 Work Item (Type 1)

 Work Item (Type 2)

Sample Kanban Board with WIP Limits

Let us understand setting up a WIP limit for a phase in Bug-fix lifecycle with an example:

Project ABC is a maintenance project with a Bug fix in scope. This project needs to set up a Kanban board with WIP limits for the flow of Bug fixes. Let us try to understand how WIP limits are set for the Impact Analysis phase.

- History data says the team is spending an average of 4 hours on impact analysis.
- There are 2 designers working in the Impact analysis phase.
- They are allocated to this project for 5 hours a day.
- The capacity for this phase is 10 hours (5*2: No of hours * No of resources).
- Hence WIP limit should be 2 for the Impact Analysis phase.

Similarly compute WIP limits for Development, Testing phases. As a preferred rule WIP limit should not be more than the number of people working on the stage.

	Backlog	Impact Analysis	Development	Testing	UAT	Done
WIP		2	4	2		
Day 1	3	2				
Day 2	1	1	1			
Day 3	2		2	1	1	
Day 4		2	0	3	0	1
Day 5	3	2	1	1	2	
Day 6	2	2	2	2	2	2
Bottleneck is visible, Designer WIP limit needs to be increased to have a better supply to Development, hence team reshuffled to increase number of designers						
Revised WIP		4	3	2		
Day 7	4	3				
Day 8	0	4	3	1		
Day 9	2	3	1	2	1	
Day 10		1	3	1	1	1
Day 11	4	1	1	2	1	2
Day 12		4	2		2	3

Red – Underutilized WIP stage

Green – 100% utilized

Illustration of WIP limits

In case the WIP limit is breached one could record them as well on the board and provide permissible limits for breaches. In case the WIP limit is breached more than the limit set, WIP limits could be re-computed.

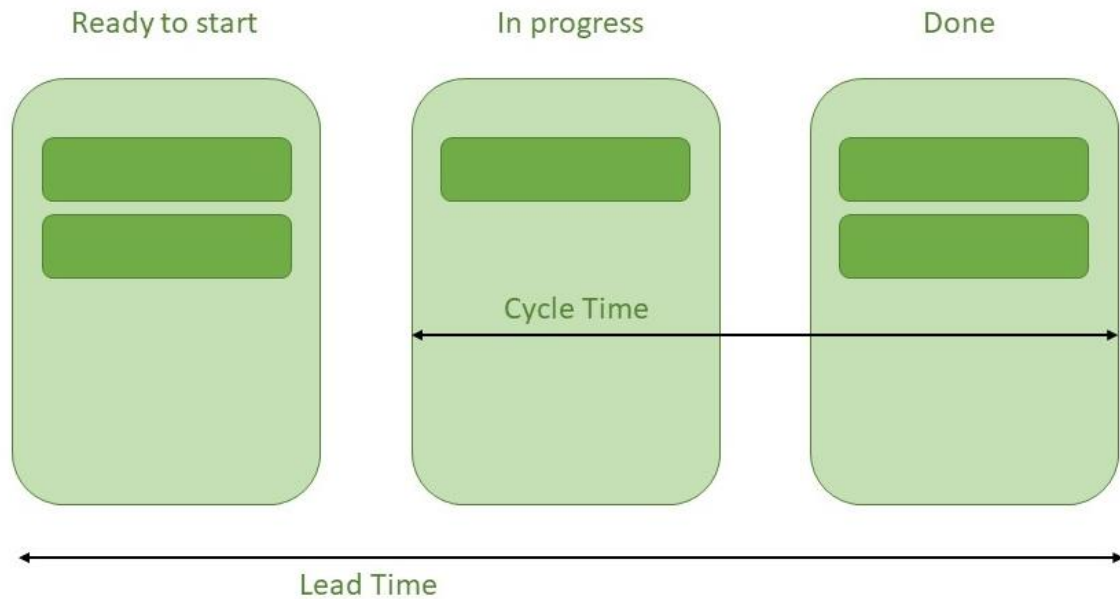
Kanban board can include Blocked tickets information also. WIP limit should be computed based on the project's context.

Step 4. Apply Pull Signals

The use of pull signals to indicate that fresh tasks are ready to be handled is an important part of a pull system. When the quantity of cards in a column falls below the specified limit in a Kanban pull system, a pull signal is generated. This tells the previous column that a new job is ready to move forward. No further tasks may be pulled once the work in progress limit has been reached unless an outstanding one has been completed first. This helps to avoid team burnout by ensuring that they only have as much work as they can handle. It also aids in the avoidance of jobs being overlooked.

Lead Time and Cycle Time

- **Lead Time:** The time span between the time a task enters the work system and the time it is completed is known as lead time. Lead time is the time it takes for an input to pass through all of the operations and arrive at the finish line. In Kanban terminology, the overall time it takes for a delegated task to reach the right-most column.
- **Cycle Time:** The cycle time displays how much time the team spends working on a prioritized task. Cycle time begins when any team member begins working on the task and transfers it to the 'in progress' column, and it continues until the task is completed.



Lead Time and Cycle Time

- **Cycle Time and Lead Time Relationship:** The most important, though often overlooked, the distinction between cycle time and lead time is their measurement units.

Cycle time is measured in amount of time per unit/process/task

Lead time is measured in elapsed time (weeks, hours, seconds)

Little's Law gives the best description of the link between Cycle time and Lead time-

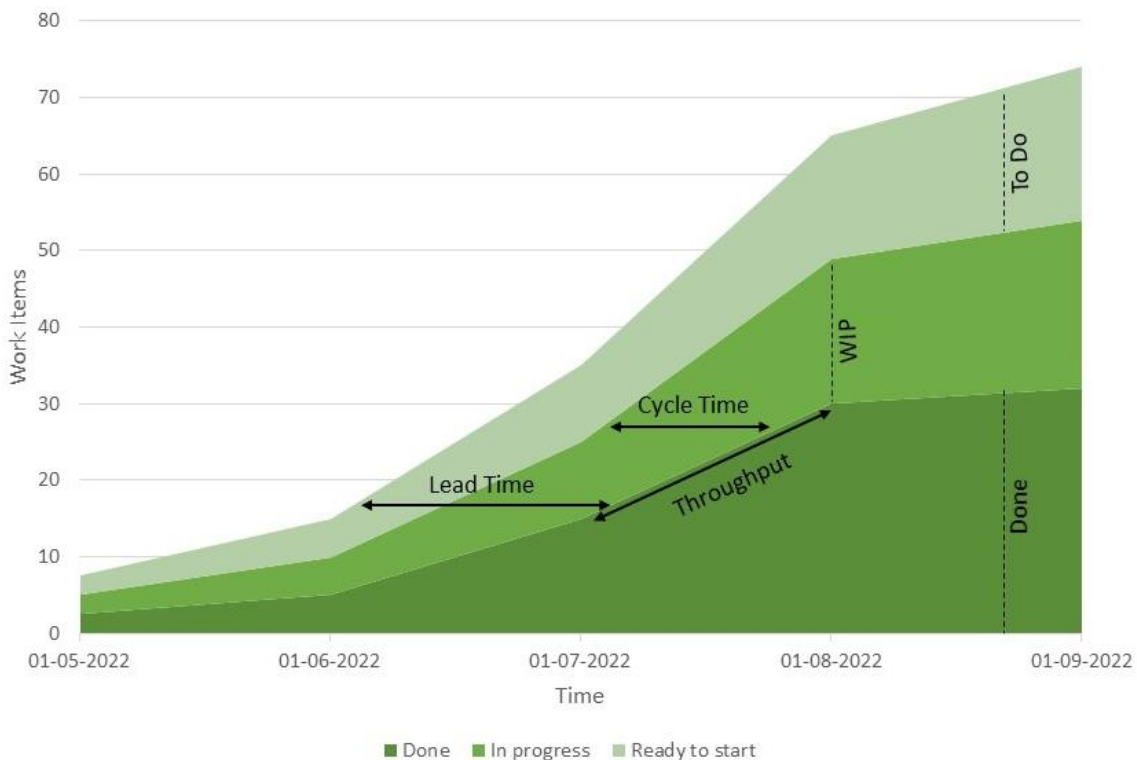
$\text{Cycle time} \times \text{WIP} = \text{Lead time (Work-In-Progress)}$

Cumulative Flow Diagram

In Kanban, a cumulative flow diagram (CFD) is an advanced analysis tool. It allows teams to see how their workflow efforts and overall project progress are being visualized. Teams can use the cumulative flow diagram to track how stable their workflow is, anticipate bottlenecks so they can alter their workflow accordingly, and make processes more predictable.

The following are three crucial parameters to look for in the CFD:

- **Cycle time:** The overall time taken by your team to accomplish each assignment from start to finish.
- **Work in progress:** This is the number of tasks that your team is currently working on.
- **Throughput:** The number of tasks your team can finish in a particular period.



Cumulative flow diagram

How to Calculate Lead Time and Cycle Time

If the correct [project management](#) software and all the information is available then calculating the lead time and cycle time of any project is easy. The Cumulative Flow Diagram (CFD) is the easiest and most widely used method of estimating both lead and cycle times.

- CFD is a graph that depicts the progress of a project by mapping it onto a graph.
- The WIP units are displayed on the vertical axis, while the time is displayed on the horizontal axis.
- The CFD is divided into segments, each of which displays a single Kanban board column.
- Planned tasks, tasks in progress, and completed tasks are the three primary divisions, just like on a Kanban board.
- However, one can add a lot more sections to further streamline the progress.

- To calculate lead time, just interpret the data from the moment a request is entered into the system (backlog), progresses through the process (in-progress), and finally reaches completion (completed).
- The Lead time is represented by the time span for this dataset.
- Calculating cycle time, on the other hand, requires skipping the first phase while the item was in the backlog and focusing on the period after work has begun.
- Cycle time is calculated as the amount of time spent per unit.

Steps of Kanban Approach

The Kanban approach involves three steps:

Step 1. Visualize: This step involves defining and visualizing the workflow.

- Understand the need for improvement.
- Define the process.
- Value stream entire process flow.
- Visualize process flow.

Step 2. Quantify: This step involves three activities:

1. Startup:

- Understand and/ or estimate WIP.
- Create initial WIP limits.

2. Adjust:

- Study the feasibility of WIP limits.
- Adjust the limits if required.
- Develop statements on the limits and policies.
- Train the team on a pilot basis.

3. Measure:

- Define current problems.
- Convert them to measurements.
- Derive metrics.
- Establish a metrics collection system.

- Define the tools used to analyze metrics and data.

Step 3. Optimize: This step involves the following three activities-

1. Identify and Improve:

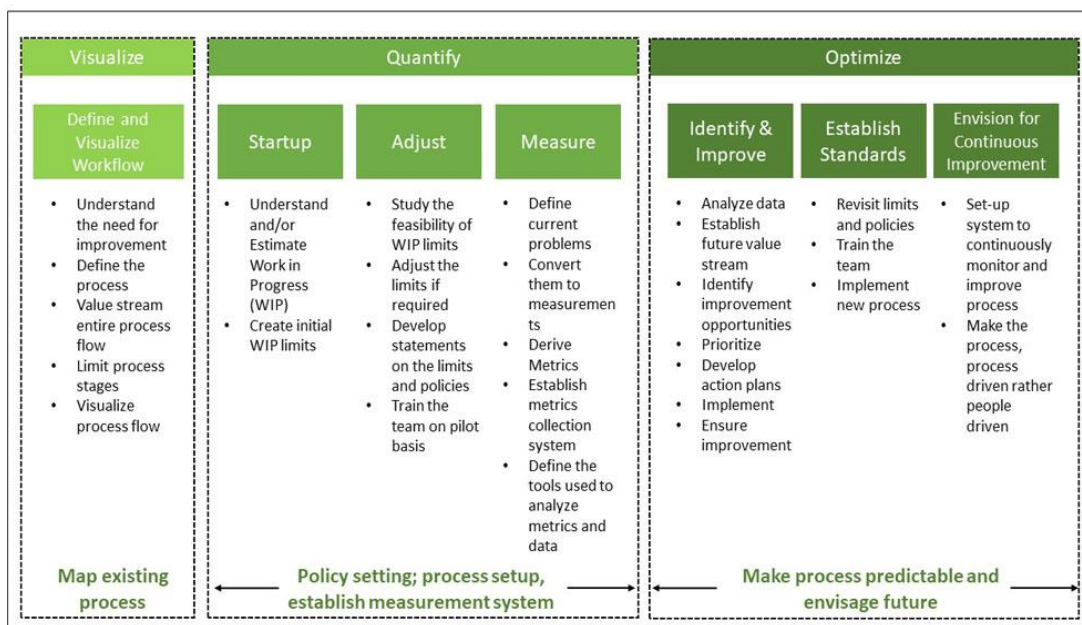
- Analyze data.
- Establish future value stream.
- Identify improvement opportunities.
- Prioritize.
- Develop action plans.
- Implement action plans.
- Ensure improvement.

2. Establish standards:

- Revisit limits and policies.
- Train the team.
- Implement a new process.

3. Envision for continuous improvement:

- Set up the system to continuously monitor and improve processes.
- Make the process, process-driven rather people-driven.



Kanban Approach

Benefits of using Kanban Framework

- Limiting Work in Progress and setting policies will result in a better focus on quality and, as a result, increased customer satisfaction.
- Use transparency to drive process improvement
- Minimized Waste
- Less process overhead
- A more precise and predictable pace guarantees that team members are never overburdened.
- Allows for quick reprioritization in order to adapt changes according to market demand.
- Better task flow
- The composition of the team can also be changed.
- Helps achieve improved productivity of teams
- Prioritization helps in streamlining processes and workflow.
- Enhanced quality of work
- Identification and elimination of bottlenecks
- Reduced queue time
- Reduction of wasted effort

Kanban vs Scrum

S No.	Kanban	Scrum
1.	Planning, release, and process improvement can have separate cadences.	Iteration is timeboxed.

S No.	Kanban	Scrum
2.	For planning and process improvement, the lead time is used as the default metric.	For planning and process improvement, Velocity is used as the default metric.
3.	Cross-functional teams are optional.	Cross-functional teams prescribed.
4.	Project Tracking: CFD can be used to understand workflow progress.	Project Tracking: Burndown chart is prescribed.
5.	WIP limited directly (per workflow state).	WIP limited indirectly (per sprint).
6.	Can add new items whenever the WIP limit falls.	Cannot add items to ongoing iteration.

DevOps Pipeline

A DevOps pipeline is a set of automated processes and tools that allows developers and operations professionals to collaborate on building and deploying code to a production environment.

DevOps is a revolutionary movement, in that it revolutionizes the siloed organizational structure that separated development and operations. The result is a cultural shift where developers and operations professionals work together, embrace automation, increase deployment speed, and are more flexible.

Continuous is a differentiated characteristic of a DevOps pipeline. This includes continuous integration, continuous delivery/deployment (CI/CD), continuous feedback, and continuous operations. Instead of one-off tests or scheduled deployments, each function occurs on an ongoing basis.

A pipeline consists of a set of tools which are classified into the following categories such as:

- Source control
- Build tools
- Containerization
- Configuration management
- Monitoring

Continuous Integration Pipeline

Continuous integration (CI) is a practice in which developers can check their code into a version-controlled repository several times per day. Automated build pipelines are triggered by these checks which allows fast and easy to locate error detection.

Some significant benefits of CI are:

- Small changes are easy to integrate into large codebases.
- More comfortable for other team members to see what you have been working.
- Fewer integration issues allowing rapid code delivery.
- Bugs are identified early, making them easier to fix, resulting in less debugging work.

Continuous Delivery Pipeline

Continuous delivery (CD) is the process that allows operation engineers and developers to deliver bug fixes, features, and configuration change into production reliably, quickly, and sustainably. Continuous delivery offers the benefits of code delivery pipelines, which are carried out that can be performed on demand.

Some significant benefits of the CD are:

- Faster bug fixes and features delivery.

- CD allows the team to work on features and bug fixes in small batches, which means user feedback received much quicker. It reduces the overall time and cost of the project.

Bottleneck

A bottleneck in DevOps is a factor that limits the efficiency or capacity of a process or system. Bottlenecks can occur at any stage of the DevOps pipeline, such as during testing, code development, deployment, integration, or delivery.

Here are some ways to identify bottlenecks in DevOps:

- **Analyze your process:** Map and examine your process flow to identify bottlenecks, which can indicate a deeper problem in the workflow's design.
- **Monitor application performance:** Use application monitoring to get a complete picture of application performance. This can help you identify bottlenecks and decide which issues need immediate attention.
- **Collect performance metrics:** DevOps metrics can help you identify and eliminate bottlenecks quickly. They can track technical capabilities and team processes.
- **Set work-in-process (WIP) limits:** WIP limits help you identify problem areas and potential bottlenecks before they cause an excess strain on your business.
- **Analyze codebase:** Once you've detected potential bottlenecks, analyze your codebase to understand the root causes. Look for common issues such as unoptimized loops, redundant database queries, or memory leaks.
- **Monitor tools:** Use DevOps monitoring tools to identify bottlenecks in the development process. They can help identify the root cause of a problem and provide insights into how to fix it.